SQL SERVER
ADVANCED CONCEPTS

A Practical Learning Guide for .NET Backend Development

CTEs • Transactions • ACID • Isolation • Stored Procedures
Functions • Views • Indexes • Query Performance

How to Use This Guide

This guide focuses on the database skills that matter when building .NET backend applications. Each section starts with the idea, then provides SQL Server syntax and practical notes.

Before You Start

- Open SQL Server Management Studio (SSMS) and connect to your SQL Server Express instance, typically .\SQLEXPRESS.
- Select the database containing your Departments, Students, Courses and Enrollments tables before running examples.
- Check actual table and column names in Object Explorer. Examples using dbo.Students assume StudentId, Name, Email, Age and DepartmentId exist.
- Some examples insert data or change isolation settings. Use a practice database and review each statement before executing it.

Learning Path

1. CTEs: make queries easier to read.
2. Transactions and ACID: keep multi-step changes consistent.
3. Isolation and locking: understand concurrent requests.
4. Stored procedures, functions and views: package reusable database logic.
5. Indexes and execution plans: measure and improve query performance.

1. Common Table Expressions (CTEs)

Concept

A Common Table Expression (CTE) is a named query result that exists only for the single statement immediately following it. It helps split a complex query into readable steps. A CTE does not automatically store data or guarantee better performance; SQL Server optimizes the overall statement.

Basic example

```
-- Find students older than the average age
;WITH AverageAge AS
(
    SELECT AVG(CAST(Age AS DECIMAL(5,2))) AS AvgAge
    FROM Students
)
SELECT StudentId, Name, Age
FROM Students
WHERE Age > (SELECT AvgAge FROM AverageAge);
```

How it works

First, SQL Server defines AverageAge. The main SELECT then uses that result to filter Students. The leading semicolon safely terminates a previous statement if needed.

CTE with aggregation and JOIN

```
-- Count students in each department
;WITH DepartmentCounts AS
(
    SELECT DepartmentId, COUNT(*) AS StudentCount
    FROM Students
    GROUP BY DepartmentId
)
SELECT d.DepartmentName, COALESCE(dc.StudentCount, 0)
```

```
AS StudentCount
FROM Departments AS d
LEFT JOIN DepartmentCounts AS dc
    ON dc.DepartmentId = d.DepartmentId
ORDER BY d.DepartmentName;
```

Recursive CTE: introduction

```
-- Generate numbers from 1 to 5
;WITH NumberSeries AS
(
    SELECT 1 AS Number
    UNION ALL
    SELECT Number + 1
    FROM NumberSeries
    WHERE Number < 5
)
SELECT Number
FROM NumberSeries
OPTION (MAXRECURSION 100);
```

Remember

Use CTEs to improve query readability, especially when a query has multiple logical steps. A CTE is not a permanent object; use a view or table when you need a reusable database object or persisted data.

2. Transactions and ACID

Concept

A transaction groups database changes into one logical unit. Either all required changes succeed, or the application can undo the changes. Transactions are essential when multiple statements must remain consistent.

Basic COMMIT and ROLLBACK

```
BEGIN TRANSACTION;

INSERT INTO Departments (DepartmentName)
VALUES (N'Artificial Intelligence');

-- If the operation is correct:
COMMIT TRANSACTION;

-- If you need to cancel instead, run ROLLBACK
-- ROLLBACK TRANSACTION;
```

Safer TRY/CATCH pattern

```
SET XACT_ABORT ON;

BEGIN TRY
    BEGIN TRANSACTION;

    INSERT INTO Departments (DepartmentName)
    VALUES (N'Data Engineering');

    -- Add related database changes here.

    COMMIT TRANSACTION;
END TRY
```

```
BEGIN CATCH
    IF XACT_STATE() <> 0
        ROLLBACK TRANSACTION;

    THROW;
END CATCH;
```

ACID explained

Atomicity: all operations happen, or none do. Consistency: constraints and rules remain valid. Isolation: concurrent transactions are controlled so they do not interfere in unacceptable ways. Durability: after a successful commit, changes are persisted according to SQL Server's durability guarantees.

Important practice

Run sample inserts with unique values because DepartmentName is UNIQUE. A repeated value will fail. Never leave a transaction open while you step away; it can hold locks and block other sessions.

3. Isolation, Locking, Blocking and Deadlocks

Concept

Isolation levels define how a transaction can see changes made by other transactions. Locks protect data while it is read or changed. Blocking happens when one session waits for another to release a resource. A deadlock occurs when sessions wait on each other in a cycle; SQL Server chooses one transaction as the victim.

Isolation levels at a glance

READ UNCOMMITTED: may read uncommitted changes (dirty reads). READ COMMITTED: prevents dirty reads and is the SQL Server default isolation level, though row-versioning database options can change how reads are implemented. REPEATABLE READ: prevents rows already read from being changed by others until the transaction ends. SERIALIZABLE: strongest locking-based level; also protects qualifying key ranges against phantoms. SNAPSHOT: reads a transactionally consistent row-versioned snapshot and requires ALLOW_SNAPSHOT_ISOLATION to be enabled.

Demonstrate READ UNCOMMITTED

```
-- Session A: make an uncommitted change
BEGIN TRANSACTION;
UPDATE Students SET Age = Age + 1 WHERE StudentId = 1;
-- Do not commit yet.

-- Session B, in another query window:
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT StudentId, Name, Age
FROM Students
WHERE StudentId = 1;
```

```
-- Back in Session A:
ROLLBACK TRANSACTION;
```

Observe blocking

```
-- Session A
BEGIN TRANSACTION;
UPDATE Students
SET Age = Age
WHERE StudentId = 1;
WAITFOR DELAY '00:00:15';
COMMIT TRANSACTION;

-- While Session A is waiting, run in Session B:
UPDATE Students
SET Age = Age
WHERE StudentId = 1;
```

Practical guidance

Keep transactions short, access tables in a consistent order, create useful indexes, and handle deadlock errors by safely retrying appropriate operations. Avoid using READ UNCOMMITTED as a blanket performance fix; dirty data can produce incorrect application results.

4. Stored Procedures

Concept

A stored procedure is a named set of SQL statements saved in the database. Applications can execute it with parameters. Procedures are useful for encapsulating common operations, centralizing database logic, and granting controlled access.

Create and execute

```
CREATE OR ALTER PROCEDURE dbo.GetAllStudents
AS
BEGIN
    SET NOCOUNT ON;

    SELECT StudentId, Name, Email, Age, DepartmentId
    FROM dbo.Students
    ORDER BY StudentId;
END;
GO

EXEC dbo.GetAllStudents;
GO
```

Procedure with a parameter

```
CREATE OR ALTER PROCEDURE dbo.GetStudentById
    @StudentId INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT StudentId, Name, Email, Age, DepartmentId
    FROM dbo.Students
    WHERE StudentId = @StudentId;
```

```
END;
GO

EXEC dbo.GetStudentById @StudentId = 1;
GO
```

Insert example with validation

```
CREATE OR ALTER PROCEDURE dbo.AddDepartment
    @DepartmentName NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    IF NULLIF(LTRIM(RTRIM(@DepartmentName)), N'') IS
NULL
        THROW 50001, 'Department name is required.',
1;

    INSERT INTO dbo.Departments (DepartmentName)
    VALUES (@DepartmentName);

    SELECT CONVERT(INT, SCOPE_IDENTITY()) AS
NewDepartmentId;
END;
GO
```

From .NET

Use parameterized commands when calling procedures from ADO.NET or EF Core. Do not build SQL by concatenating user-provided values. In SSMS, GO is a client batch separator and is not sent to SQL Server as part of the procedure definition.

5. SQL Functions

Concept

A function returns a value or a table and can be used inside queries. Scalar functions return one value. Inline table-valued functions return a table expression. Functions have restrictions compared with stored procedures and are best used for computations or reusable query expressions.

Scalar function

```
CREATE OR ALTER FUNCTION dbo.GetAgeCategory
(
    @Age INT
)
RETURNS NVARCHAR(20)
AS
BEGIN
    RETURN
        CASE
            WHEN @Age IS NULL THEN N'Unknown'
            WHEN @Age < 18 THEN N'Minor'
            WHEN @Age < 60 THEN N'Adult'
            ELSE N'Senior'
        END;
END;
GO

SELECT Name, Age, dbo.GetAgeCategory(Age) AS
AgeCategory
FROM dbo.Students;
GO
```

Inline table-valued function

```
CREATE OR ALTER FUNCTION dbo.GetStudentsByDepartment
(
    @DepartmentId INT
)
RETURNS TABLE
AS
RETURN
(
    SELECT StudentId, Name, Email, Age, DepartmentId
    FROM dbo.Students
    WHERE DepartmentId = @DepartmentId
);
GO

SELECT *
FROM dbo.GetStudentsByDepartment(1);
GO
```

Procedure vs function

A procedure is executed with EXEC and can perform actions such as inserts or updates. A function is used as part of an expression or query and returns a value/table; user-defined functions have restrictions on side effects. Choose the simplest object that matches the job.

6. Views

Concept

A view is a saved SELECT statement that behaves like a virtual table. It can simplify repeated joins and provide a controlled projection of columns. A normal view does not store its result rows separately.

Create a view

```
CREATE OR ALTER VIEW dbo.StudentDepartmentView
AS
SELECT
    s.StudentId,
    s.Name AS StudentName,
    s.Email,
    s.Age,
    d.DepartmentName
FROM dbo.Students AS s
INNER JOIN dbo.Departments AS d
    ON d.DepartmentId = s.DepartmentId;
GO

SELECT *
FROM dbo.StudentDepartmentView
ORDER BY StudentName;
GO
```

Why use a view?

Views can make complex joins easier for application developers, expose only approved columns, and create a stable query interface. Permissions should still be configured correctly. A view does not automatically make a query faster; indexed views are a special feature with additional requirements.

7. Indexes

Concept

An index is a data structure that helps SQL Server locate rows without scanning every row in a table. Indexes can speed up reads but consume storage and add work to INSERT, UPDATE and DELETE operations.

Clustered and nonclustered

A clustered index defines how table rows are organized at the leaf level of the index; a table can have only one clustered index. A nonclustered index is a separate structure containing indexed key values and row locators, and a table can have multiple nonclustered indexes.

Create a nonclustered index

```
-- Useful when queries frequently filter by
DepartmentId
CREATE NONCLUSTERED INDEX IX_Students_DepartmentId
ON dbo.Students (DepartmentId);
GO

-- Example query that may benefit
SELECT StudentId, Name, Age
FROM dbo.Students
WHERE DepartmentId = 2;
GO
```

Composite index

```
-- Consider only if workload frequently filters by
department
-- and then orders or filters by age.
CREATE NONCLUSTERED INDEX IX_Students_DepartmentId_Age
ON dbo.Students (DepartmentId, Age);
GO
```

Index design tips

Index columns used frequently in selective WHERE predicates, joins, and important ORDER BY patterns. The leading key order matters for composite indexes. Do not add indexes blindly. Check existing indexes and the actual workload before creating or dropping them.

8. Query Performance and Optimization

Execution plan

An execution plan shows how SQL Server intends to retrieve and process data. In SSMS, choose Include Actual Execution Plan (Ctrl+M), then execute the query. Compare estimated and actual row counts and inspect expensive operators. A scan is not always bad; for a small table or a query returning many rows, scanning may be the best choice.

Compare IO and time

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT StudentId, Name, Age
FROM dbo.Students
WHERE DepartmentId = 2;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```

Prefer searchable predicates

```
-- Often less index-friendly because a function is
applied to the column:
-- WHERE YEAR(AdmissionDate) = 2026

-- Prefer a range predicate (assuming AdmissionDate
exists):
-- WHERE AdmissionDate >= '20260101'
--   AND AdmissionDate <  '20270101'
```

Practical optimization checklist

1. Select only needed columns. 2. Filter early and accurately. 3. Use correct data types and parameterized queries. 4. Check join predicates and row

counts. 5. Inspect actual execution plans and STATISTICS IO/TIME. 6. Add or adjust indexes only when evidence supports it. 7. Test with representative data. 8. Measure again after each change.

Common misconception

An index does not guarantee a faster query. A poor index, a nonselective filter, outdated statistics, parameter sensitivity, or a query returning most of a table can make an index seek unnecessary or even less efficient.

9. Quick Revision Cheat Sheet

CTE

```
;WITH RecentStudents AS
(
    SELECT StudentId, Name, Age
    FROM dbo.Students
    WHERE Age >= 18
)
SELECT * FROM RecentStudents;
```

Transaction

```
BEGIN TRY
    BEGIN TRANSACTION;
    -- INSERT / UPDATE / DELETE statements
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;
```

Stored procedure

EXEC dbo.GetStudentById @StudentId = 1;

Function

SELECT dbo.GetAgeCategory(21) AS AgeCategory;

View

SELECT * FROM dbo.StudentDepartmentView;

Index

```
CREATE NONCLUSTERED INDEX IX_Students_Age
ON dbo.Students (Age);
```

Common mistakes

• Forgetting the WHERE clause in UPDATE/DELETE. • Leaving a transaction open. • Assuming CTEs are persisted tables. • Using string concatenation to build SQL. • Creating indexes without checking workload. • Comparing performance using only elapsed time instead of also examining reads and execution plans.

Final Practice Checklist

- Write a CTE that finds students older than the average age.
- Use a transaction to insert a department and roll it back safely.
- Explain dirty reads, blocking and deadlocks in your own words.
- Create and execute a stored procedure with an input parameter.
- Create a scalar function and an inline table-valued function.
- Create a view joining Students and Departments.
- Create a nonclustered index and explain which query may benefit.
- Compare actual execution plans and STATISTICS IO before and after a change.
- Explain why every index has a write/storage cost and why actual workload matters.

Note: Examples are teaching templates. Verify your schema, permissions, SQL Server options and data before executing them in a real environment.